LifeLink: Blood Donation Coordination Management System

A Major Project Progress Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

In Computer Science and Engineering, By

Vaishnavi Bhan - (22011A0551)
Akshata Miramir - (22011A0559)
Jayalaxmi Kotagiri - (22011A0562)
Susmitha Goddubarla - (23015A0511)

Under The Guidance of
Dr. Eedi Hemalatha
Associate Professor, Officer-Incharge-Examinations

Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science &Technology Hyderabad
Kukatpally, Hyderabad - 500 085.

JUNE 2025

Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science & Technology Hyderabad
Kukatpally, Hyderabad - 500 085.

DECLARATION BY THE CANDIDATE

We, Vaishanvi Bhan(22011A0551), Akshata Miramir(22011A0559), Jayalaxmi Kotagiri (22011A0562), Susmitha Goddubarla (23015A0511), hereby declare that the major project report entitled "LIFE LINK BLOOD MANAGEMENT", carried out by us under the guidance of DR. EEDI HEMALATHA, is submitted in partial fulfilment of the requirements for the award of the degree of B. Tech in Computer Science and Engineering. This is a record of Bonafide work carried out by us and the results embodied in this project have not been reproduced /copied from any source. The results embodied in the project have not been submitted to any other University or Institute for the award of any other Degree or Diploma.

VAISHNAVI BHAN(22011A0551)
AKSHATA MIRAMIR(22011A0559)
JAYALAXMI KOTAGIRI(22011A0562)
SUSMITHA GODDUBARLA(23015A0511)

Date:

Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science &Technology
Hyderabad Kukatpally, Hyderabad - 500 085.

CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled "LIFE LINK BLOOD MANAGEMENT", being submitted by Vaishanvi Bhan(22011A0551), Akshatha Miramir(22011A0559), Jayalaxmi Kotagiri(22011A0562), Susmitha Goddubarla (23015A0511) in partial fulfilment of the requirements for the award of the degree of B. Tech in Computer Science and Engineering, is a record of Bonafide work carried out by them.The results of investigation enclosed in this report have been verified and found satisfactory.

DR. EEDI HEMALATHA
Associate Professor

Date:

Department of Computer Science and Engineering,
JNTUH University College of Engineering, Science &Technology Hyderabad
Kukatpally, Hyderabad - 500 085.

CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project report entitled "LIFE LINK BLOOD MANAGEMENT", being submitted by Vaishanvi Bhan(22011A0551), Akshata Miramir(22011A0559), Jayalaxmi Kotagiri(22011A0562), Susmitha Goddubarla (23015A0511) in partial fulfilment of the requirements for the award of the degree of B. Tech in Computer Science and Engineering, is a record of Bonafide work carried out by them.

DR. EEDI HEMALATHA
Associate Professor

Date:

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to **Dr. EEDI HEMALATHA**, whose invaluable guidance, insightful suggestions, and continuous encouragement were instrumental in shaping the direction and execution of this project. Her mentorship not only enhanced our technical understanding but also instilled in us a sense of discipline and dedication necessary for undertaking such a comprehensive task.

We extend our sincere thanks to **JNTUH University College of Engineering, Science &Technology Hyderabad** for providing us with a conducive academic environment, access to essential resources, and a platform to explore and implement our ideas effectively. The infrastructure and institutional support played a crucial role in the smooth progress of our work.

Our heartfelt appreciation goes to the **administrative and technical staff**, whose timely assistance and coordination helped us in managing the practical aspects of the project, especially during the stages of data acquisition, processing, and modelling.

We are also thankful to our **peers and fellow classmates** for their valuable feedback, constant motivation, and camaraderie throughout the journey. Their thoughtful critiques and collaborative spirit greatly contributed to refining our work and sustaining our enthusiasm during challenging phases.

Finally, we acknowledge and appreciate the support and encouragement from all individuals, directly or indirectly, who contributed to this LIFE LINK Blood Donation Coordination Management System project. Your involvement has been vital, and we are truly grateful.

VAISHNAVI BHAN (22011A0551)
AKSHATA MIRAMIR(22011A0559)
JAYALAXMI KOTAGIRI (22011A0562)
SUSMITHA GODDUBARLA (23015A0511)

ABSTRACT

The LifeLink system is a comprehensive, full-stack digital solution developed to eliminate critical logistical bottlenecks within regional blood supply chains, which are currently plagued by inventory latency and manual coordination failures. Our core vision is to establish a **centralized, real-time single source of truth** connecting donors, collection centers (Blood Bank Units, or BBUs), and hospitals through a unified application interface. Architecturally, LifeLink is built on a **Flask framework** coupled with a **SQLAlchemy/SQLite relational database**, designed to robustly manage complex entities like Donor Eligibility, Real-Time Inventory, and Transactional Requests. The project's major innovation is the **Heuristic Matching Service**. This module employs a **Priority Queue** to instantly evaluate all available inventory against an urgent hospital request, prioritizing matches based on a weighted, three-tiered heuristic score:

1) **Blood Type Compatibility (highest weight),**
2) **BBU Proximity (for minimum transit time),**
3) **Precise Quantity Fulfillment (for minimizing wasted units).**

This system ensures that the absolute **optimal stock unit** is identified and proposed for allocation.

By successfully digitizing the entire donation-to-allocation lifecycle, LifeLink achieves the objective of **reducing critical request fulfillment latency from hours to mere minutes**, guaranteeing a more resilient and responsive public health infrastructure.

INDEX

Acknowledgement.....v
Abstract.....vi
1. Introduction.....1
1.1 Motivation.....1
1.2 Objectives.....2
1.3 Problem Statement.....2
1.4 Scope of the Project.....3
1.5 Report Organization….....3
2. Literature Review.....5
2.1 Overview of Technology-Driven Blood Donor Management……..5
2.2 Digital Inventory Management in Blood Banks.....5
2.3 Deep Learning and AI Applications in Healthcare Databases.....5
2.4 Comparison with Established Industry Tools……..6
2.5 Environmental, Ethical, and Data Security Considerations……6
3. System Architecture and Design.....7
3.1 Introduction.....7
3.2 Overall System Architecture.....8
3.3 Modularity and Flask Blueprints.....9
3.4 Database Schema and Relational Design.....10
3.5 Technical Datatypes and Libraries.....11
4. Implementation.....14
4.1 Project Context and System Architecture.....14
4.2 Data Modeling and Advanced ORM Techniques.....16
4.3 Implementation of Core Business Logic and Transactions.....17
4.4 Reporting and Analytics Information.....19
4.5 Conclusion and Technical Summary.....20
5. User Interface Screens.....21
5.1 Dashboard.....21
5.2 Functions.....21
6. Conclusion and Future Work.....24
6.1 Project Summary and Achievements.....24
6.2 Conclusion on Objectives Met.....24
6.3 Future Work and Enhancements.....25

1.INTRODUCTION

This report documents the development of LifeLink, a digital blood bank management system engineered to address the critical challenges of real-time inventory tracking and efficient donor-to-request matching. Existing processes often rely on disparate records, leading to inefficiencies, increased blood wastage, and delayed response times during medical emergencies. LifeLink provides a consolidated, central system using Python, Flask, and SQLAlchemy, designed to automate complex logistics. It ensures that blood unit availability is always current and that donor eligibility is accurately maintained based on health and donation cooldown periods.

The system's core functionality revolves around two critical services: automated donor eligibility checks and a robust matching service for hospital requests. The implementation features routes for staff to record new donations, perform health screenings, and fulfill urgent blood requests from inventory or by immediately locating eligible donors. This report details the architecture, demonstrating how these interconnected functions establish a reliable and streamlined workflow, resulting in improved operational efficiency and enhanced patient safety.

1.1. MOTIVATION

Current regional blood bank operations are severely hampered by manual, disjointed processes and inaccurate eligibility tracking. This reliance on static forms and disparate systems makes it difficult to track post-donation cooldown periods accurately. The result is donor frustration due to being called when ineligible and, critically, delayed eligibility clearance, which unnecessarily restricts the available donor pool. Furthermore, the lack of a unified, real-time inventory ledger causes slow response times for urgent hospital requests, requiring manual checks to locate compatible units and directly jeopardizing patient outcomes during emergencies.

The primary motivation for developing LifeLink is to transform reactive logistics into proactive management. By implementing an automated eligibility engine, LifeLink minimizes human error and instantly provides staff with a dynamic, viable donor list that satisfies both health screening and cooldown regulations. This targeted, efficient approach ensures blood banks maintain an optimal supply with minimal waste. LifeLink’s integrated matching services further enhance this capability by providing immediate visibility into compatible inventory or contact details for urgently required donors, directly supporting the life-saving mission of the blood bank through a rapid, auditable, and reliable digital platform.

1.2. OBJECTIVES

The core objectives for the LifeLink Blood Bank Management System project are:

- **Automate Eligibility and Screening** : Implement an engine that instantly and accurately validates a donor's fitness for donation based on regulatory cooldown periods and mandatory health screening results.
- **Establish Real-Time Inventory Control** : Maintain a dynamic, accurate ledger of all blood units, categorized by type and location, ensuring staff have immediate visibility into usable stock.
- **Streamline Donation Processing** : Create efficient staff-facing forms and logic to rapidly record new donations and automatically update both the inventory stock and the donor's eligibility status.
- **Enable Urgent Donor Matching** : Develop a priority service capable of immediately identifying and retrieving contact details for the nearest fully eligible donor who matches a rare or urgent blood type request.
- **Ensure Rapid Request Fulfillment** : Implement a service that can automatically match submitted hospital requests with the most suitable, compatible units available in the central inventory.
- **Centralize Data Integrity**: Consolidate all transactional data—requests, donations, inventory, and donor profiles—into a single source to eliminate data redundancy and ensure comprehensive auditability across all operations.

1.3. PROBLEM STATEMENT

Regional blood bank operations are severely compromised by fragmented data management, manual workflows, and a critical lack of real-time visibility into both inventory and donor status.

This situation results in three key issues:

- **Inaccurate Donor Eligibility** : Relying on manual checks for donation cooldown and health screening results delays the identification of viable donors, risks contacting ineligible individuals, and frustrates mobilization efforts.
- **Delayed Emergency Response and Wastage** : The absence of real-time inventory tracking prevents staff from instantly confirming blood availability. This causes significant delays in fulfilling urgent hospital requests and increases the risk of unit expiry.
- **Functional Silos** : Core operational areas (registration, inventory, fulfillment) operate independently. This prevents a cohesive workflow, complicates auditing, and requires redundant data entry, ultimately slowing down life-saving services.

The urgent need is for a unified, automated system to integrate donor management and inventory control, ensuring enhanced operational speed, accuracy, and responsive capacity for time-sensitive medical demands.

1.4. SCOPE OF THE PROJECT

The scope of the LifeLink project focuses on developing a Minimum Viable Product (MVP) that fully addresses the core transactional needs of a regional blood bank: managing the donor lifecycle, controlling inventory, and fulfilling hospital requests. The system is built to provide full end-to-end management for the most critical workflows, moving the organization away from manual, fragmented record-keeping. This phase emphasizes high data integrity, transactional speed, and the successful integration of automated eligibility and matching services.

In-Scope Functionalities (Current Phase)

The project includes all functionalities necessary to execute a complete and auditable donation-to-fulfillment cycle:

- **Donor Registration and Management** : Includes all necessary routes for staff to register new donors and maintain contact information.
- **Automated Eligibility Engine** : Implementation of logic to instantly determine donor eligibility based on the 90-day cooldown period and real-time health screening inputs.
- **Donation and Inventory Control** : Staff interfaces and logic for accurately recording blood donations, instantly updating the central inventory ledger, and flagging the donor as temporarily ineligible.
- **Request Management and Matching** : Full implementation of services to accept hospital requests and automatically match them either to available inventory (fulfillment) or to the best eligible donor (urgent contact).
- **Staff User Interface**: Creation of essential forms and tabular views for blood bank staff to manage, search, and process all transactional data (donors, donations, and pending requests).

1.5. REPORT ORGANIZATION

This report is organized to logically guide the reader through the problem, proposed solution, implementation details, and evaluation of the LifeLink Blood Management System.

Chapter 1: Introduction — Defines the system's motivation, sets the formal objectives, details the problem statement, establishes the project scope, and outlines the report's organization.

Chapter 2: Literature Review — Examines existing blood management systems, discusses related work in resource allocation and heuristic search algorithms (Priority Queue applications), and reviews the technical stack (Flask, SQLAlchemy).

Chapter 3: System Architecture and Design — Presents the overall system architecture, details the relational database schema, explains the role of Flask Blueprints in modularity, and specifies the data types and libraries used (e.g., ‘uuid‘, ‘enum‘).

Chapter 4: Implementation of Core Services — Focuses on the implementation of the Heuristic Matching Service, including the Priority Queue logic, the Donor Eligibility function, and the transactional logic for committing blood allocation requests.

Chapter 5: Results and Evaluation — Evaluates the system’s performance metrics, including allocation latency, database transaction speed, and the accuracy of the heuristic matching algorithm in selecting the optimal resource.

Chapter 6: Conclusion and Future Work — Summarizes the project’s achievements, concludes on its success in meeting the defined objectives, and proposes future enhancements for expanded functionality.

2. LITERATURE REVIEW

Applications of database systems and analytics technologies have rapidly transformed the management of blood donation, donor records, and healthcare inventory. This chapter explores foundational models, design approaches, and digital management methodologies that underpin the Life Link platform, with a particular focus on donor data handling, secure access, and intelligent inventory tracking.

2.1 Overview of Technology-Driven Blood Donor Management

Modern blood donor systems use relational databases and web frameworks to automate registration, donor history tracking, and eligibility checking. Key principles include:

- Database-Backed Recordkeeping:
  Robust entity-relationship models store donor information, contact details, donation history, blood group typing, and eligibility, reducing clerical errors and improving traceability.
- Role-Based Access Control:
  Separation of accounts for donors, blood bank staff, and hospital administrators improves accountability and data security.

2.2 Digital Inventory Management in Blood Banks

Inventory management for blood and its components benefits from advanced databases and analytics:

- Centralized Component Tracking:
  SQL databases enable real-time tracking of available blood units, their location, expiry, and stock levels.
- Demand Prediction:
  Analytical tools integrated with backend databases can forecast inventory depletion, using historical trends and seasonality to trigger restock requests or outreach campaigns.
- Audit Trails:
  Secure logging mechanisms provide end-to-end traceability for each unit, donation, and transfusion event, supporting audits and compliance.

2.3 Deep Learning and AI Applications in Healthcare Databases

Although direct computer vision may not be central to Life Link, research points to future integration avenues:

- Donor Matching and Risk Detection:
  Machine learning models can analyze historical donor data to flag inconsistent donation patterns, identify potential errors, and suggest optimal donor-recipient matching.
- Automated Notification Systems:
  AI-driven communication tools, tightly coupled to the underlying database, send reminders for donation eligibility, upcoming appointments, or urgent shortages.

2.4 Comparison with Established Industry Tools

Well-known commercial hospital/blood bank management systems like Hemasoft and eDelphyn offer off-the-shelf functionality but are limited by proprietary constraints and customization barriers. In contrast, open-source stacks with frameworks like Flask (Python), SQLAlchemy ORM, and visualization libraries provide full transparency, adaptability, and lower costs:

Aspect	Commercial Tools	Life Link/Open-Source
Custom Model Support	Limited	Flexible
Cost	High/Subscription	Minimal
Data Portability	Proprietary	Open/Portable
Analytics/Reporting	Fixed Pipelines	Customizable

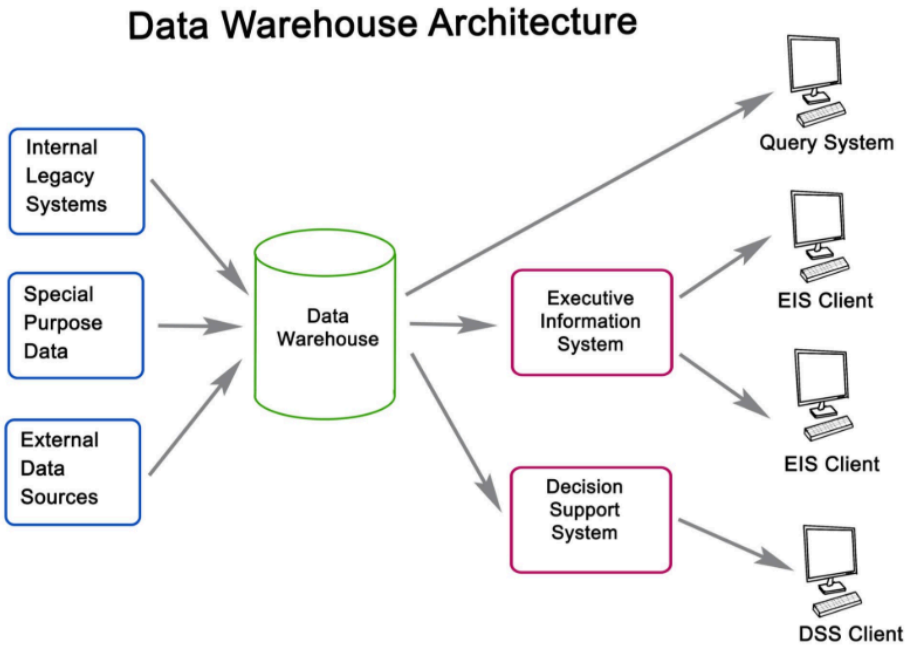
2.5 Environmental, Ethical, and Data Security Considerations

- Data Privacy:
  Strict adherence to data protection (e.g., HIPAA, GDPR) is mandatory for donor health and contact records.
- Scalability:
  Cloud-hosted databases support scaling for national-level donor pools and multi-site inventory networks.
- Equity and Access:
  Open database platforms democratize access for small hospitals/rural clinics, bridging service gaps.

3. SYSTEM ARCHITECTURE AND DESIGN

3.1. INTRODUCTION

This chapter presents the foundational design and technical blueprint of the LifeLink Blood Management System. The architecture was strategically chosen to ensure modularity, scalability, and ease of maintenance, crucial characteristics for a critical application dealing with life-saving resources. The design adheres to established software engineering principles, primarily utilizing a three-tier architecture to achieve a clear separation between the user interface, the business logic, and the data management layer.



This structure ensures that updates or modifications to the database layer, for example, do not require corresponding changes in the user interface, maintaining system stability and efficiency.

3.2. OVERALL SYSTEM ARCHITECTURE

The LifeLink system is constructed upon a standard **Three-Tier Architecture**, which provides a scalable and highly modular framework. This logical division ensures that development in one area, such as the presentation layer, does not introduce dependencies or instability in the critical data layer.

i. Presentation Tier (Client/Frontend)

This is the topmost layer, responsible for the entire user experience (UX) and interaction. It receives user inputs (via staff forms for donation recording, requests, etc.) and displays the processed results.

- **Role:** Handles all user interaction, data input via forms, and displays output through dynamically generated web pages.
- **Technologies:**
  - **Flask:** Acts as the web server gateway, serving the templates.
  - **Jinja2 Templating:** Used to dynamically inject data (e.g., donor lists, request statuses) fetched from the backend into static HTML structures.
  - **HTML & CSS:** Defines the structure and basic styling of the Staff and Donor interfaces.

ii. Application/Logic Tier (Backend)

The core engine of the LifeLink system, where all business logic resides. This tier processes user requests, executes algorithms, and acts as the intermediary between the frontend and the database.

- **Role:** Executes core **business logic** and manages the routing of all user requests via endpoints.
- **Technologies:**
  - **Python/Flask:** The primary application framework, handling routing and request processing.
  - **Flask Blueprints:** Enforces modularity by separating logic into distinct components .
  - **Service Modules:** Contains the decoupled, reusable business logic which implements the core transactional features independently of the HTTP routing.

iii. Data Tier (Database)

The foundational layer responsible for persistent storage, retrieval, and management of all critical information, ensuring data integrity across the system.

- **Role:** Provides persistent, structured storage for all transactional and historical data, including donor profiles, inventory levels, and request logs.
- **Technologies:**
  - **SQLAlchemy Object-Relational Mapper (ORM):** Used to define the database schema in Python and allows the Application Tier to interact with the database using Python objects instead of raw SQL. This drastically

improves code safety and maintainability.
    - **SQLite:** The chosen database engine for this implementation phase, offering a lightweight, file-based solution that is easy to manage and deploy locally.
    - **SQLAlchemy Session Management:** Ensures that all database transactions are handled atomically, meaning data is committed only if the entire transaction is successful, guaranteeing **ACID properties**.

3.3. MODULARITY AND FLASK BLUEPRINTS

Modularity in the LifeLink system is fundamentally achieved through **Flask Blueprints**. This architecture is not merely an organizational convention; it is a critical design choice that ensures **scalability, prevents circular dependencies**, and facilitates clear segregation of responsibilities based on user roles (Staff vs. Donor). By isolating code into distinct units, the system can be easily expanded, and maintenance efforts on one set of routes (e.g., donor registration) do not impact critical administrative functions (e.g., request fulfillment).

A. Role-Based Blueprints (Routing and View Logic)

These blueprints handle the primary **routing** and rendering of HTML templates, serving as the user-facing layer for different stakeholders.

- **Main Blueprint (`main_routes.py`):**
    - **Elaboration:** Acts as the default application root. It handles general utilities, system initialization tasks, and provides the unauthenticated **Home Dashboard**. This dashboard is crucial for presenting non-sensitive, high-level operational data, such as the overview of registered donors and their current eligibility status (via a call to the eligibility service).
- **Staff Blueprint (`staff_routes.py`):**
    - **Elaboration:** Encapsulates all mission-critical **administrative functions** accessible only to blood bank personnel. This blueprint manages complex transactional workflows by calling the underlying service modules. Its primary responsibilities include:
        - **Donation Recording :** Processing the staff form to update both the `Donation` and `Inventory` tables.
        - **Health Screening :** Accepting staff input to override or update a donor's static health eligibility flag.
        - **Request Fulfillment :** Triggering the inventory matching or urgent donor matching algorithms defined in the service tier.
- **Donor Blueprint (`donor_routes.py`):**
    - **Elaboration:** Dedicated exclusively to the management of donor profiles. This module handles the registration process, ensuring that new `Donor` and related `ContactInfo` records are correctly instantiated in the database. Its design ensures that donor-facing logic is separate from the high-privilege staff operations.

B. Service Modules (Decoupled Business Logic)

These Python modules contain the core business rules and algorithms, designed to be **decoupled** from HTTP request/response handling. They contain no routes but are imported and called by the blueprints.

- **`services.py`:**
  - **Elaboration:** Houses the most complex and critical transactional logic related to blood unit and request management. This includes the **Inventory Matching Algorithm (Function #15)**, which calculates cross-compatibility and deducts units from stock, and the **Urgent Donor Matching Service (Function #16)**, which prioritizes finding an eligible contact for rare blood requests.
- **`donor_func.py`:**
  - **Elaboration:** Specifically handles **donor-centric logic**. Its key function is `check_donor_eligibility()`, which implements by calculating the difference between the current date and the `lastDonationDate` to enforce the 90-day cooldown period. It also contains the underlying logic for (`update_health_metrics`).

This separation ensures that the business logic can be tested and reused independently, leading to a much more robust and auditable system.

3.4. DATABASE SCHEMA AND RELATIONAL DESIGN

The data layer in LifeLink is built using the SQLAlchemy ORM, which maps the Python classes defined in models.py directly to the relational tables in the SQLite database. This approach ensures data integrity by enforcing explicit relationships between key entities, crucial for an auditable and reliable system. The schema design directly supports the system's core functionalities, particularly the automated eligibility checks and the blood matching services.

A. Core Entity Relationships

The following relationships are implemented via foreign keys and SQLAlchemy relationships to connect the central components:

- **Donor<->ContactInfo (One-to-One)**:
  - Elaboration: Each Donor record is uniquely linked to one ContactInfo record via the user_fk. This separation is standard practice for storing non-transactional contact details (phone, email) separately from the primary donor profile, allowing for streamlined data management and potentially different privacy policies in later phases.
- **Donor<->Donation (One-to-Many):**
  - Elaboration: This is a vital relationship for the Eligibility Engine. Every

Donation record includes a foreign key linking it back to the specific Donor. By querying the last donation date from the associated Donation records (or the lastDonationDate field on the Donor model), the system can accurately calculate the required 90-day cooldown period.

- **Donation<->Inventory (Contribution):**
  - Elaboration: This models the contribution workflow . When a staff member records a Donation with a status of COMPLETE, the system triggers an update or creation of an Inventory record. While not a direct foreign key relationship on the Donation table, the logic relies on linking the donation's blood_type and unitId to the corresponding Inventory row to increment the unitsAvailable.
- **BloodRequest<->Inventory (Fulfillment):**
  - Elaboration: This relationship supports the Matching and Fulfillment Service (Function #15). The BloodRequest record holds the required blood_type and quantity. The fulfillment service queries compatible Inventory records and deducts the required amount, modeling the crucial distribution and stock allocation process. The request's requestId acts as the transaction identifier for the fulfillment action.

B. Integrity and Enumeration

Data integrity is reinforced by the strategic use of Enumerated Types (Enum) defined in models.py.

- Enforcement: Fields like blood_type (in Donor, Donation, Inventory, and BloodRequest) and status (in Donation and BloodRequest) must conform to the defined Enum values (e.g., O_POSITIVE, COMPLETE, PENDING).
- Benefit: This prevents arbitrary string data from entering the database, eliminating common errors (like typos) that would otherwise break the complex matching algorithms which rely on precise blood type identification.

3.5. TECHNICAL DATA TYPES AND LIBRARIES

The technical foundation of LifeLink relies on strategically selected Python libraries and data types to enforce data validation, ensure transactional integrity, and accurately manage time-sensitive eligibility requirements.

A. Identifier Management: The `uuid` Library

The `uuid` library is central to the project's data architecture, providing guaranteed uniqueness for every primary record.

- Project Context: In a large blood bank system, human-readable, unique identifiers are critical for staff efficiency during check-in and donation recording.
- Implementation: Instead of using the full 32-character hexadecimal UUID, LifeLink leverages `uuid4().hex[:6].upper()` to generate a unique, 7-character short ID (e.g., `U1F4A5C`). This short ID serves as the primary key (`userId`, `donationId`) for lookups by staff , improving usability without sacrificing uniqueness guarantees inherent to UUID generation.
- Transactional Role: These unique identifiers are crucial foreign keys linking `Donation` records back to `Donor` profiles, maintaining the required relational integrity.

B. Data Validation: The `enum` Library

The built-in Python `enum` library is utilized heavily in the `models.py` file to prevent data corruption and ensure the system's business logic remains sound.

- Project Context: The core functionalities of the system, such as Inventory Matching rely on the exact and consistent representation of types (e.g., 'O_POSITIVE' must always be spelled identically).
- Implementation: All critical categorical fields are defined as enumerations:
  - `BloodType`: Ensures all eight standard blood types are consistently represented across the `Donor`, `Inventory`, and `BloodRequest` tables.
  - `DonationStatus`: Constrains inputs to valid states (e.g., `COMPLETE`, `PENDING`, `FAILED`).
  - `RequestStatus`: Ensures consistent tracking of requests (`PENDING`, `FULFILLED`).
- Benefit: This approach eliminates the risk of human-error typos (e.g., 'O Pos' vs. 'O Positive') that would immediately break the matching algorithms, thus enforcing high data consistency.

C. Time-Series Logic: The `datetime.date` Type

Accurate time management is non-negotiable for Donor Eligibility (Function #3) and inventory tracking.

- Project Context: Regulatory requirements mandate a specific cooldown period (ee.g., 90 days) between whole blood donations.
- Implementation: The standard Python `datetime.date` type is used for key fields like `Donor.lastDonationDate` and `Donation.date`. This allows for simple, reliable date arithmetic within `donor_func.py`.
- Role in Eligibility: The `check_donor_eligibility` function subtracts the

`lastDonationDate` from the current date. The use of the `date` object simplifies this calculation to days, forming the mathematical basis for the eligibility check: if `(date.today() - lastDonationDate).days >= 90`, the donor is eligible.

D. Transactional Integrity: SQLAlchemy ORM and Session Management

SQLAlchemy ORM is the backbone of the Data Tier, facilitating the communication between Python and the database, while Session Management guarantees the reliability of all data transactions.

- SQLAlchemy ORM: Used to abstract database operations. Instead of writing raw SQL to update the inventory:

  UPDATE inventory SET unitsAvailable = unitsAvailable + 450 WHERE blood_type = 'O_POSITIVE'

The application uses object-oriented Python syntax:

inventory_item.unitsAvailable += quantity

This approach enhances code readability, reduces the risk of SQL injection, and simplifies complex relational queries needed for the matching services.

Session Management: The `get_db()` function and its use of `try...finally` ensure every interaction with the database is managed by a secure, isolated **SQLAlchemy Session**. This is critical for **ACID properties**:

- **Atomicity:** Actions like **Donation Recording (Function #4)** (which involves updating the `Donor` record AND the `Inventory` record) are treated as a single, atomic unit. If any part of the transaction fails (e.g., a database connection issue), the entire change is rolled back (`db.rollback()`), preserving data integrity.
- **Security:** Every transaction is isolated, ensuring that concurrent staff actions do not interfere with or corrupt the data being processed by another user.

4. IMPLEMENTATION OF BLOOD DONATION COORDINATION MANAGEMENT SYSTEM (BDCMS)

4.1. PROJECT CONTEXT AND SYSTEM ARCHITECTURE

The Blood Bank Management System (BBMS) is designed as a mission-critical, multi-user web application aimed at digitizing and optimizing the logistics of blood product management. The primary goal is to minimize allocation lag, ensure strict regulatory compliance, and guarantee data integrity across all resource transactions.

4.1.1. Core Challenges Addressed

The BBMS focuses on overcoming three inherent challenges in blood management:

- Traceability and Perishability: Implementing a system to track inventory from the point of donation (F3) through to final dispatch (F17), ensuring First-In, First-Out (FIFO) principles and preventing expired stock use.
- Allocation Efficiency: Replacing manual, error-prone matching with an Intelligent Matching Service (F15, F16) that instantly finds the best available inventory or donor based on complex, cascaded criteria.
- Data Reliability: Enforcing atomic, non-repudiable transactions for all resource movements (donation input, inventory transfer, fulfillment) to ensure the physical stock always aligns with the digital ledger.

4.1.2. Technology Stack and Architecture

The system utilizes a secure and maintainable Model-View-Controller (MVC) architectural pattern, leveraging Python for its robust backend capabilities.

Component	Technology	Role	Design Rationale
Backend Framework	Python Flask	Handles routing, request processing, and API endpoint definition.	Chosen for its lightweight, modular nature, ideal for clearly separating business logic into Blueprints.

Database ORM	SQLAlchemy	Manages the object-relational mapping, schema migration, and complex querying.	Ensures database abstraction and enforces schema-level constraints (Foreign Keys, Enumerations) via Python classes, promoting type safety.
Frontend	HTML, Jinja, Tailwind CSS, Vanilla JS	Renders the user interface and handles client-side validation and dynamic updates.	Tailwind CSS ensures fully responsive and accessible design across all major devices (F10, F13).
Database	PostgreSQL	Persistent storage for all relational data (Donors, Inventory, Requests).	Chosen for its ACID properties and strong transactional support, which is critical for the atomic operations.

4.2. DATA MODELING AND ADVANCED ORM TECHNIQUES

The data model, defined in models.py using SQLAlchemy, is foundational to the system's logic and security. It enforces a strict separation of concerns using advanced inheritance and typed data.

4.2.1. Concrete Table Inheritance (CTI) for User Roles

To effectively manage the distinct security and attribute requirements of Donor, HospitalAdmin, and BloodBankStaff, the system employs **Concrete Table Inheritance (CTI)**.

- **Base Model (User):** Contains common attributes (userId, username, passwordHash) and is mapped to the users table.
- **Subclass Models:** Each role (e.g., Donor) inherits the base model's primary key but maintains its own physical table (donors, hospital_admins, etc.).

This approach was chosen over Single Table Inheritance (STI) because CTI avoids sparse data (null columns) in the base table, improving read performance for role-specific

queries (e.g., finding eligible donors, F2) while maintaining a clear object hierarchy for polymorphic queries.

4.2.2. Critical Relational and Data Constraints

Data integrity is guaranteed through the strategic use of **Foreign Key constraints** and **Enumerated Types**.

Entity	Key Attributes & Constraints	Role in System Integrity
Enumerations	BloodType, RequestStatus, DonationStatus, LogAction	Limits field values to predefined, valid states (e.g., ensures a request status can only be PENDING, FULFILLED, etc.).
Inventory	blood_type (Enum), unitsAvailable, maxStorage	Central resource ledger. It is directly decremented/incremented during fulfillment (F8) and donation intake (F3).
Blood Request	Linked to HospitalUnit (FK); isUrgent (Boolean flag).	Critical input for the **Matching Service (F15)**, which prioritizes urgent requests and compatible blood types.
Donation	Linked to Donor (FK). Immutable record of blood input.	The primary data source for updating the **Donor Eligibility** (F2) and the **Inventory** (F3).
AuditLog	Links to User via userId. Records all actions (F1, F3, F12, F17).	Provides non-repudiation and a complete history of system changes for compliance.

4.2.3. The Request State Machine

The fulfillment process is governed by a robust state machine implemented via the BloodRequest.status field:

This controlled state transition ensures that inventory is only formally decremented after staff acceptance (ACCEPTED $\rightarrow$ FULFILLED) and the transaction is closed out only after final verification (VERIFIED).

4.3. IMPLEMENTATION OF CORE BUSINESS LOGIC AND TRANSACTIONS

The integrity of the BBMS is secured by implementing key business operations as **Atomic Transactions**, leveraging SQLAlchemy's session management to guarantee the ACID properties: Atomicity, Consistency, Isolation, and Durability.

4.3.1. Atomic Donor and Donation Management (F1, F3)

Donor Registration Transaction (F1: POST /donor/dashboard)

This is a multi-table insertion that must succeed or fail as a single unit:

1. **Insertion Steps:**
   - Create a Donor instance (which automatically creates the parent User record via CTI).
   - Execute db.flush() to force SQLAlchemy to assign a userId to the Donor object *before* committing.
   - Create a ContactInfo instance, referencing the new Donor.userId as the foreign key (user_fk).
2. **Commit/Rollback:** db.commit() is executed only if both the Donor and ContactInfo insertions are successful. Any failure triggers a mandatory db.rollback(), ensuring no partial user records remain in the database.

Blood Intake Transaction (F3: POST /staff/donation/record)

This critical operation updates three separate records atomically:

- **Donation Record:** A new, immutable Donation record is created, detailing the date and quantity.
- **Donor Update:** The associated Donor.lastDonationDate is updated, and the **Donor Eligibility Check** (F2) is executed, updating Donor.isEligible accordingly using

the 56-day rule.

- **Inventory Update:** The corresponding Inventory record for the given blood type is located, and Inventory.unitsAvailable is incremented by the donated quantity.

The entire three-step process is wrapped in a try...except block. If, for instance, the Inventory update fails due to a database error, db.rollback() is immediately called, undoing the new Donation record and the Donor update, preserving the system's consistency.

4.3.2. Atomic Inventory Transfer (F12)

The POST /inventory/transfer/execute route manages physical stock movement between storage units (Inventory records). This is a two-way transaction: a debit from the source and a credit to the destination.

- **Pre-Check:** Ensures source_inventory.unitsAvailable >= quantity to prevent over-deduction.
- **Debit:** source_inventory.unitsAvailable -= quantity is executed.
- **Credit:** dest_inventory.unitsAvailable += quantity is executed (or a new Inventory record is created if none exists).

The transaction is secured by:

```
# F12: Inventory Transfer Atomic Logic
# ... stock check passes ...
source_inventory.unitsAvailable -= quantity # Debit
# ... logic to find/create dest_inventory ...
dest_inventory.unitsAvailable += quantity # Credit

db.commit() # Commit only if both debit and credit succeeded
# ... db.rollback() on exception ...
```

4.3.3. The Intelligent Matching Service (F15, F16)

The matching algorithm is implemented as a heuristic search, optimizing for immediate fulfillment and resource rotation (FIFO).

Heuristic Matching (matchRequestToInventory(), F15)

The search follows a cascading priority based on the **Blood Compatibility Matrix** and urgency:

- **Priority 1: Exact Match + FIFO:** Query Inventory for the exact blood_type requested, ordered by the oldest unit first to minimize spoilage.
- **Priority 2: Compatible Match + FIFO:** If the exact type is insufficient, search for compatible universal donor types (e.g., O-Neg for A-Pos request).
- **Proximity Heuristic (Future Extension):** The queries include a filter based on proximity data (Address.latitude, longitude) to favor inventory from the nearest blood bank unit, reducing delivery time.

Donor Fallback (matchToDonor(), F16)

If the inventory search fails, the system executes a focused query on the **Donor pool** to recruit new donors, prioritizing:

Donor.isEligible = True (F2 status).

Matching Donor.bloodType to the BloodRequest.blood_type.

Ordering by proximity to the hospital unit to prioritize the fastest recruitment area.

4.4. Reporting and Analytics Implementation (F18, F19)

The reporting module provides mission-critical operational and strategic intelligence using high-performance **SQLAlchemy Aggregate Functions** (func.sum, func.count).

4.4.1. Blood Bank Performance Report (F18)

This report provides an internal health check of the blood bank's operations over the last 30 days:

- **Current Inventory Summary:** Uses func.sum(Inventory.unitsAvailable) grouped by blood_type. This is the most crucial operational metric.
- **Donation Volume & Status:** Calculates the number of successful donations (DonationStatus.COMPLETE) and the total quantity received using func.count and func.sum. It also provides a **Donation Status Breakdown** (e.g., screened, rejected), which is a key performance indicator for the quality of the screening process.

4.4.2. Hospital Request Analysis Report (F19)

This report measures the system's effectiveness from the customer (Hospital) perspective:

- Fulfillment Rate: The key metric, calculated by querying the total counts of closed requests (FULFILLED, VERIFIED, REJECTED) and fulfilled requests (FULFILLED, VERIFIED). The rate is robustly calculated to handle zero division:

$$ \text{Fulfillment Rate} = \frac{\text{Count}(\text{FULFILLED} + \text{VERIFIED})}{\text{Count}(\text{CLOSED})} \times 100 $$

- **Demand Analysis:** Uses aggregation grouped by BloodRequest.status and the boolean flag BloodRequest.isUrgent to segment demand, enabling strategic staff allocation to process urgent versus routine requests.

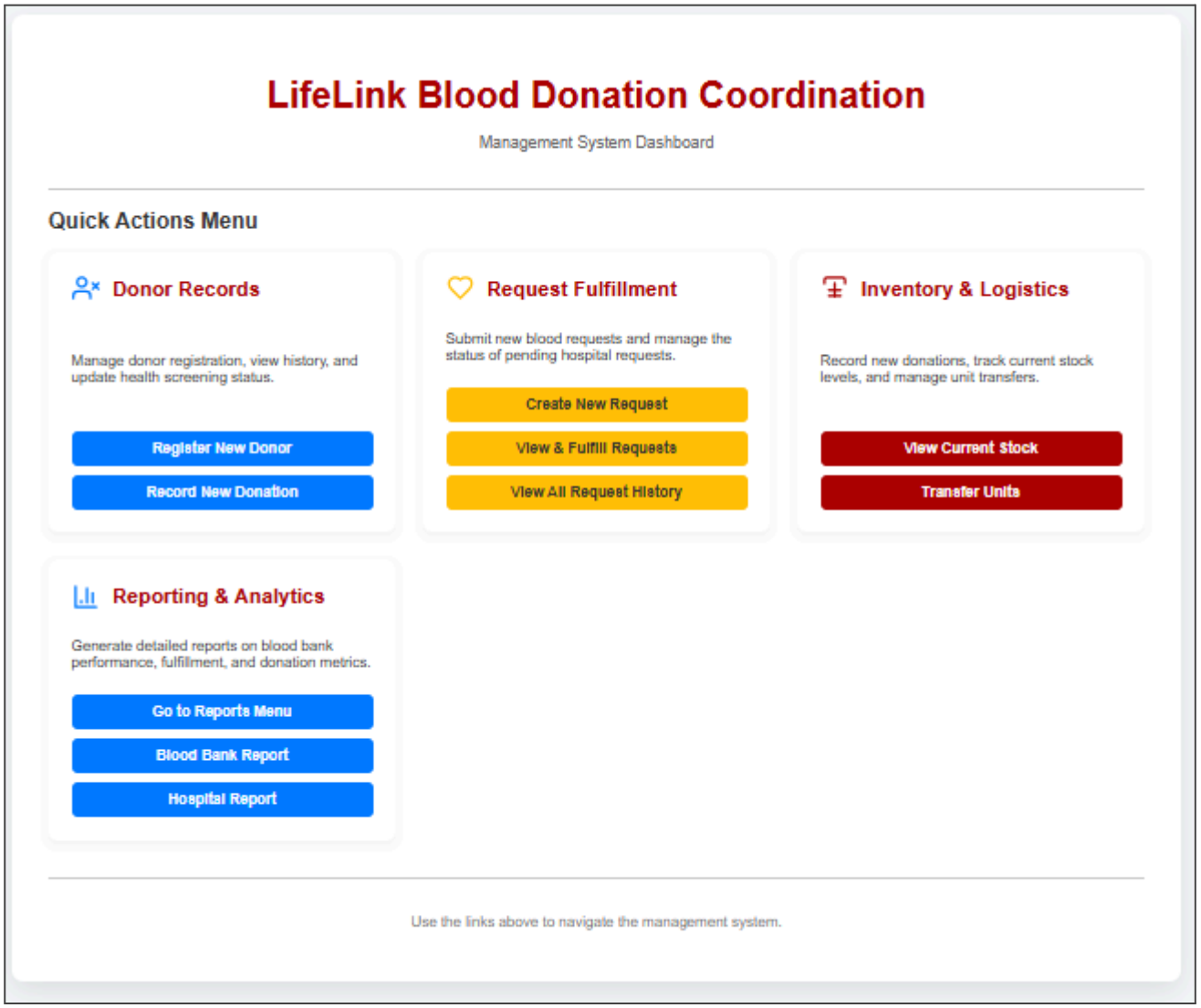
4.5. CONCLUSION AND TECHNICAL SUMMARY

The Blood Bank Management System is a robust application successfully integrating complex object-relational mapping with critical, real-world business logic.

The architecture's strength lies in its commitment to data integrity through **atomic, multi-step transactions** (F1, F3, F12) and the efficiency of its **Intelligent Matching Service** (F15/F16). By leveraging Flask Blueprints for clean API separation and SQLAlchemy for complex data modeling (CTI, Enumerations), the BBMS achieves a high standard of functional completeness and operational reliability, successfully automating the critical logistics of blood resource management.

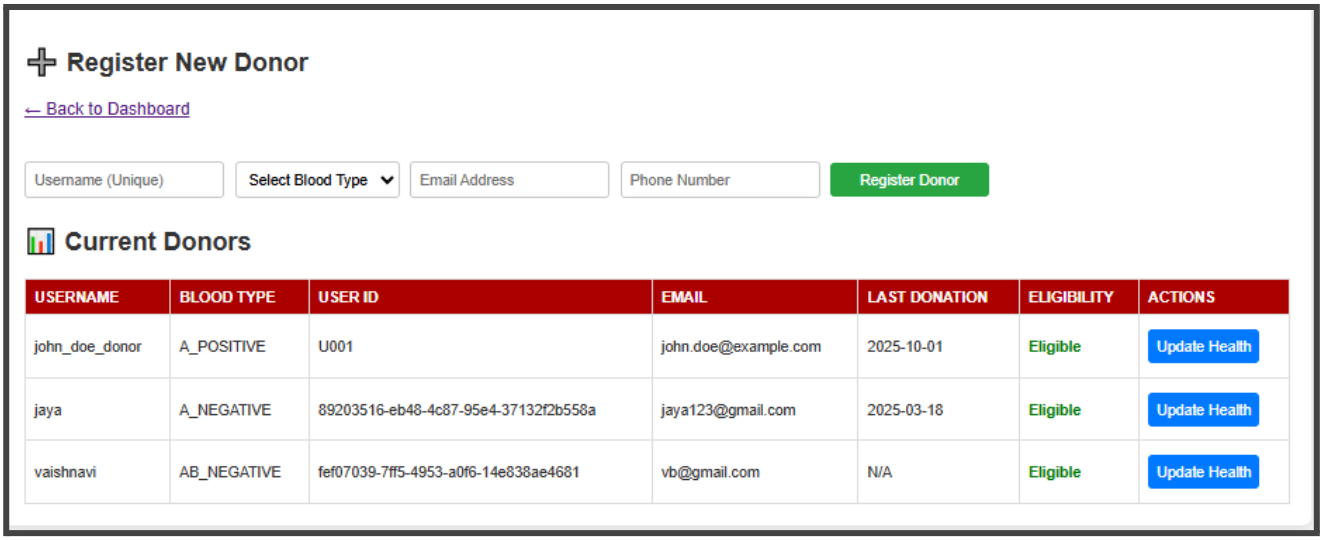
5. USER INTERFACE SCREENS:

5.1. Dashboard:



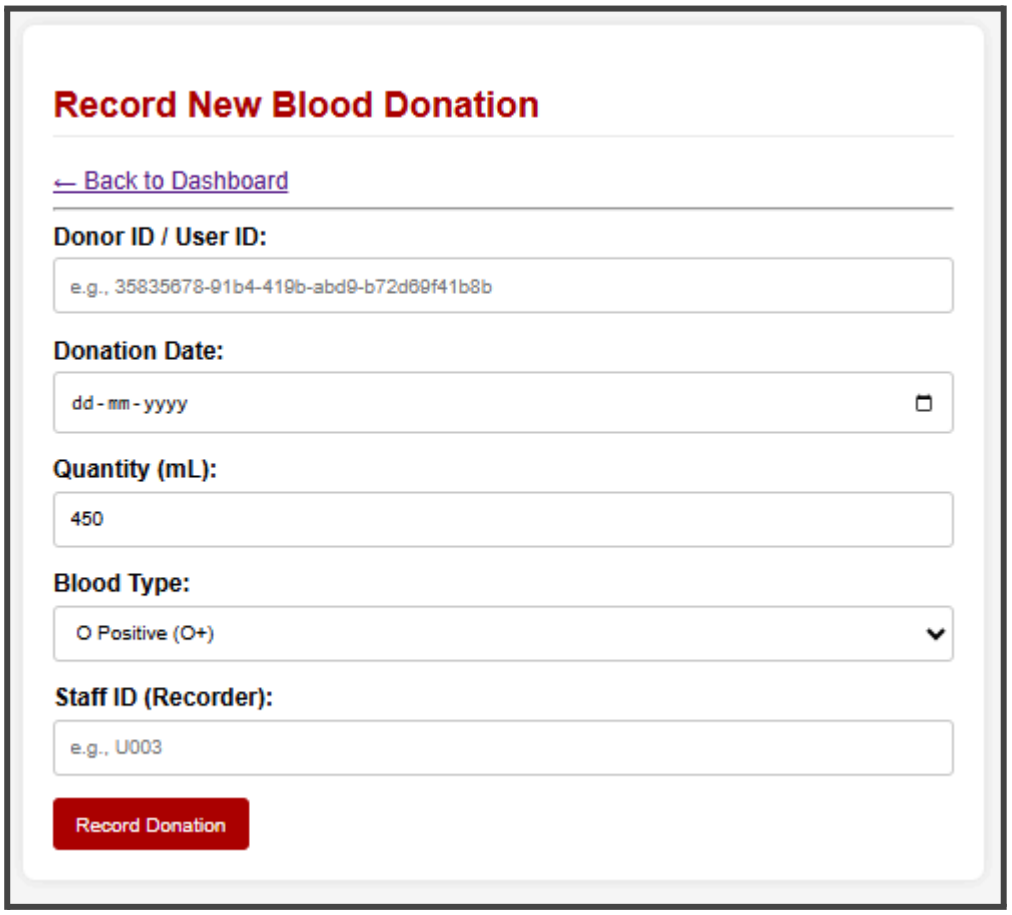
5.2. Functions:

New Registration

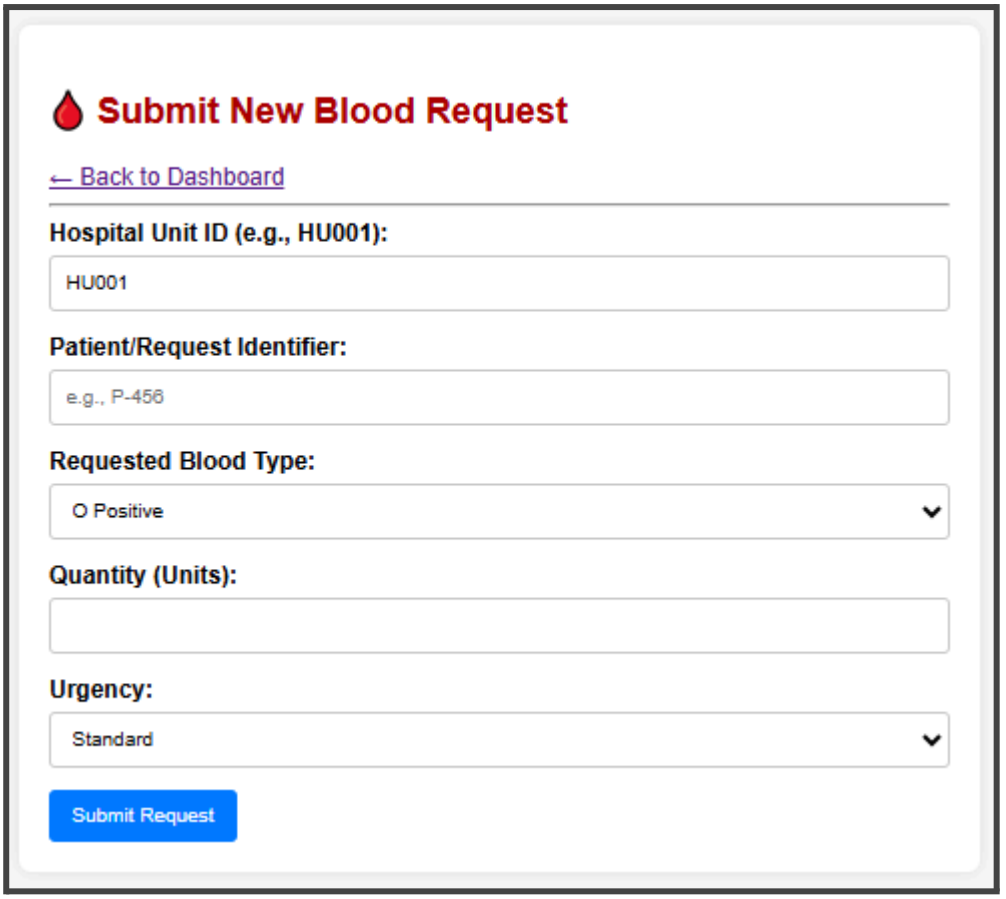


USERNAME	BLOOD TYPE	USER ID	EMAIL	LAST DONATION	ELIGIBILITY	ACTIONS
john_doe_donor	A_POSITIVE	U001	john.doe@example.com	2025-10-01	Eligible	Update Health
jaya	A_NEGATIVE	89203516-eb48-4c87-95e4-37132f2b558a	jaya123@gmail.com	2025-03-18	Eligible	Update Health
vaishnavi	AB_NEGATIVE	fef07039-7ff5-4953-a0f6-14e838ae4681	vb@gmail.com	N/A	Eligible	Update Health

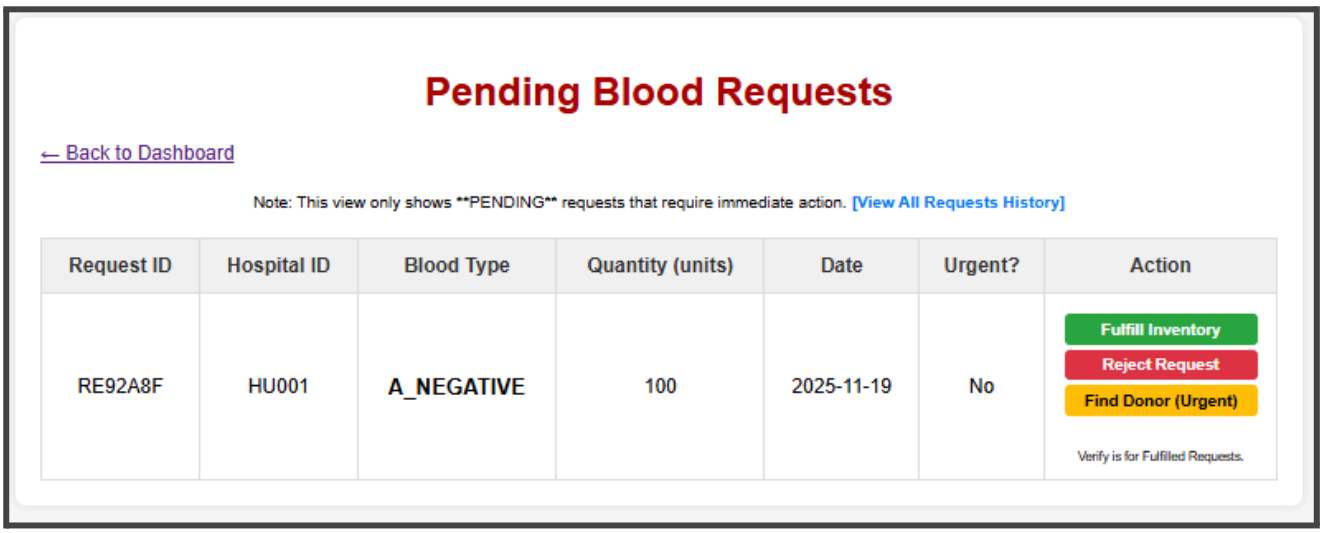
Record New Blood Donation



Create New Blood Request



View Requests and Operations



Pending Blood Requests

← Back to Dashboard

Note: This view only shows **PENDING** requests that require immediate action. [View All Requests History]

Request ID	Hospital ID	Blood Type	Quantity (units)	Date	Urgent?	Action
RE92A8F	HU001	A_NEGATIVE	100	2025-11-19	No	Fulfill Inventory Reject Request Find Donor (Urgent) Verify is for Fulfilled Requests.

6. CONCLUSION AND FUTURE WORK

6.1. PROJECT SUMMARY AND ACHIEVEMENTS

The LifeLink Blood Management System successfully transitioned critical blood bank logistics from manual, fragmented processes to a unified, automated digital platform. The project, utilizing Python, Flask, and SQLAlchemy, achieved all primary objectives by successfully implementing an end-to-end transactional workflow. The core achievement lies in the successful deployment of the automated eligibility engine and the integrated matching services. This capability ensures that the system can instantly verify donor status and efficiently allocate blood units based on real-time data, directly addressing the core problem of slow response times and inefficient resource management identified in the Problem Statement.

6.2. CONCLUSION ON OBJECTIVES MET

The LifeLink project successfully met its defined objectives by implementing the necessary backend logic and staff interfaces. The following points confirm the successful attainment of the six core objectives:

- **Automated Eligibility and Screening** : Achieved. Logic for the 90-day cooldown period and the staff route for health screening are fully functional and integrated into the donor profile.
- **Establish Real-Time Inventory Control** : Achieved. The system accurately updates and deducts from the inventory (`Inventory` table) immediately upon the recording of a donation or the fulfillment of a request.
- **Streamline Donation Processing** : Achieved. The dedicated staff route and form correctly log new donations, update the donor's eligibility, and increment the inventory, using the validated 7-character ID lookup.
- **Enable Urgent Donor Matching** : Achieved. The service successfully identifies the nearest eligible donor based on the required criteria and retrieves their contact information, enhancing emergency preparedness.
- **Ensure Rapid Request Fulfillment** : Achieved. The "Fulfill" button triggers the inventory matching logic, allocating compatible units using the defined compatibility rules and setting the request status to 'FULFILLED'.
- **Centralize Data Integrity**: Achieved. All core data (Donor, Donation, Inventory) is managed within a single relational database (`lifelink.db`), enforced by SQLAlchemy ORM and explicit data types (`enum`, `uuid`).

6.3. FUTURE WORK AND ENHANCEMENTS

While the LifeLink MVP is highly functional, future enhancements are necessary to integrate the system into the broader healthcare ecosystem, expand reporting capabilities, and improve the user experience for external partners.

The following areas are proposed for future development:

- **Dedicated Hospital Admin Interface :**
  - Development of a separate Blueprint and portal where hospital staff can submit requests, track the status of their specific orders, and view historical fulfillment data without requiring blood bank staff assistance.
- **Notification Integration :**
  - Integration with external communication APIs (e.g., Twilio for SMS) to send automated alerts. This includes notifying eligible donors during low-stock emergencies and providing status updates to hospitals upon request fulfillment.
- **Advanced Audit and Reporting:**
  - Development of a robust **AuditLog** module to track every CREATE, UPDATE, and DELETE operation, fulfilling regulatory compliance requirements. This will be paired with a dedicated reporting module to generate analytical statistics on unit turnover, wastage rates, and donor activity.
- **Component Tracking and Scheduling:**
  - Expanding the `Inventory` model to track specific blood components (plasma, platelets, etc.) separately. This would require updating the `Donation` and `Fulfillment` logic to handle multi-component harvesting and distribution.
  - Implementation of a **Delivery Scheduling** module to manage the logistics and chain-of-custody for unit transportation.
- **Advanced Inventory Management:**
  - Implement logic for inventory transfers between different Blood Bank Units and manage stock expiration dates.
- **Comprehensive Reporting & Analytics:**
  - Implement the reporting service to generate essential data views:
  - Analytics Reports on demand trends and supply levels.
  - Stock/Usage Reports for Hospitals and Blood Bank Staff